

Resumen del código `index.html` – Ejemplo Hit Test con WebXR

Este programa implementa una aplicación de **Realidad Aumentada (AR)** utilizando la API **WebXR**. Su objetivo es detectar superficies del mundo real mediante la técnica denominada **Hit Test**, mostrar una retícula indicando el punto donde puede colocarse un objeto virtual y, cuando el usuario toca la pantalla, posicionar un modelo tridimensional de una flor sobre dicha superficie.

El archivo está compuesto por una estructura HTML básica y un módulo JavaScript que contiene toda la lógica de la aplicación. Durante la ejecución, el navegador solicita acceso a una sesión de Realidad Aumentada, configura el entorno gráfico utilizando WebGL y comienza a recibir información de la cámara y de los sensores del dispositivo. A partir de esta información es posible calcular la posición y orientación del teléfono en el espacio, así como detectar superficies horizontales o verticales donde pueden colocarse objetos virtuales.

La primera parte del documento corresponde a la estructura HTML. En la sección `<head>` se definen diversos metadatos necesarios para que la aplicación funcione correctamente en dispositivos móviles. Entre ellos se encuentra la codificación UTF-8, la configuración del viewport y las opciones que permiten ejecutar la aplicación como una experiencia de pantalla completa en Android e iOS. También se cargan los archivos CSS que proporcionan el estilo visual de la página.

En la sección `<body>` únicamente existe un encabezado con una breve descripción del ejemplo y un enlace para regresar al índice de los ejemplos oficiales. El resto del funcionamiento se encuentra implementado dentro de un bloque `<script type="module">`, lo que permite utilizar módulos de JavaScript mediante la instrucción `import`.

El programa importa varias clases auxiliares. La clase `WebXRButton` crea el botón que permite iniciar y finalizar la sesión de Realidad Aumentada. La clase `Scene` administra todos los objetos que serán dibujados en pantalla. El ***Renderer establece la comunicación entre WebXR y WebGL para generar los gráficos.*** La clase `Node` representa un objeto dentro de la escena, mientras que `Gltf2Node` permite cargar modelos tridimensionales almacenados en formato GLTF. Finalmente, `DropShadowNode` crea una sombra bajo el objeto virtual para mejorar la percepción de profundidad.

Después de importar las clases, el programa declara varias variables globales. Estas variables almacenan referencias a la sesión XR, al espacio de coordenadas utilizado por la aplicación, a la fuente encargada de realizar las pruebas de impacto y al contexto WebGL empleado para el renderizado. También se crean los objetos principales de la escena, entre ellos un nodo que contendrá la flor, la propia flor cargada desde un archivo GLTF, la retícula que indica el punto de impacto y una sombra situada debajo del modelo.

Inicialmente la flor permanece invisible para evitar que aparezca flotando en el espacio antes de que el usuario seleccione una superficie válida. La retícula también comienza oculta y únicamente será visible cuando el sistema detecte una superficie donde sea posible colocar el objeto.

La función `initXR()` constituye el punto de inicio de la aplicación. Su responsabilidad consiste en crear el botón que permitirá iniciar la experiencia de Realidad Aumentada y comprobar si el navegador ofrece soporte para sesiones del tipo `immersive-ar`. Si el dispositivo no soporta WebXR, el botón permanecerá deshabilitado evitando que el usuario intente iniciar una sesión incompatible.

Cuando el usuario presiona el botón **START AR**, se ejecuta la función `onRequestSession()`. Esta función solicita al navegador la creación de una sesión WebXR indicando que requiere dos características fundamentales: un sistema de coordenadas local (`local`) y la capacidad de realizar pruebas de impacto (`hit-test`). Si el navegador acepta la solicitud, devuelve una sesión XR que será utilizada durante toda la experiencia.

La función `onSessionStarted()` configura completamente la sesión recién creada. En primer lugar registra los eventos necesarios para detectar cuándo la sesión termina y cuándo el usuario toca la pantalla. Posteriormente crea el contexto WebGL compatible con WebXR y construye la capa gráfica (`XRWebGLLayer`) donde serán dibujados todos los objetos virtuales.

Una vez inicializado el sistema gráfico, el programa solicita dos espacios de referencia. El primero corresponde al espacio del observador (`viewer`), utilizado exclusivamente para lanzar los rayos que buscan superficies mediante Hit Test. El segundo corresponde al espacio local (`local`), que representa el sistema de coordenadas estable donde vivirán todos los objetos de la escena.

Después de obtener ambos espacios de referencia, el programa crea una fuente de pruebas de impacto (`XRHitTestSource`). Esta fuente será utilizada continuamente para lanzar un rayo desde la posición de la cámara hacia el entorno real con el propósito de encontrar superficies detectables.

Al finalizar la configuración, el programa solicita el primer cuadro de animación utilizando `session.requestAnimationFrame()`. A partir de este momento comienza el ciclo principal de la aplicación.

El corazón del programa es la función `onXRFrame()`, ejecutada aproximadamente sesenta veces por segundo mientras la sesión permanece activa. En cada iteración se obtiene la posición actual del usuario mediante `frame.getViewerPose()`. Si existe una fuente de Hit Test disponible, el programa solicita los resultados correspondientes al cuadro actual mediante `frame.getHitTestResults()`.

Cuando se detecta al menos una superficie, el primer resultado obtenido contiene una transformación espacial que indica exactamente dónde se encuentra el punto de impacto. Esa transformación se asigna a la retícula, permitiendo que ésta siga continuamente la superficie detectada mientras el usuario mueve el teléfono. Si no existe ninguna superficie válida, la retícula permanece oculta.

Después de actualizar la posición de la retícula, el programa inicia el renderizado de la escena, solicita el siguiente cuadro de animación y finalmente dibuja todos los objetos utilizando la información proporcionada por WebXR. Este ciclo se repite de forma continua hasta que el usuario abandona la sesión.

La interacción del usuario se realiza mediante el evento `select`, generado cuando el usuario toca la pantalla. La función `onSelect()` comprueba primero si la retícula se encuentra visible. Si existe una superficie válida, copia la matriz de transformación de la retícula hacia el nodo que contiene la flor. Como consecuencia, la flor aparece exactamente en la posición donde anteriormente se encontraba la retícula.

La función `moveARObjectTo()` únicamente recibe una matriz de transformación y la asigna al objeto virtual. Además cambia la propiedad `visible` para permitir que la flor sea dibujada durante los siguientes cuadros de animación. Este procedimiento no crea nuevos objetos; simplemente desplaza el único modelo existente hacia una nueva ubicación.

Cuando el usuario decide finalizar la experiencia, la función `onEndSession()` cancela la fuente de Hit Test y solicita el cierre de la sesión XR. Posteriormente el evento `end` ejecuta `onSessionEnded()`, función encargada de actualizar el estado del botón para indicar que ya no existe una sesión activa.

En conjunto, este ejemplo constituye una demostración sencilla pero completa del funcionamiento básico de WebXR. A través de poco más de un centenar de líneas de código muestra cómo inicializar una sesión de Realidad Aumentada, crear un contexto gráfico compatible con WebXR, detectar superficies del mundo real mediante Hit Test, representar visualmente el punto de impacto utilizando una retícula y colocar un modelo tridimensional cuando el usuario interactúa con la pantalla. Aunque carece de funciones avanzadas como anclas, persistencia o manipulación de múltiples objetos, proporciona una base sólida para comprender el flujo de trabajo de una aplicación de Realidad Aumentada desarrollada directamente sobre la API WebXR.

Código QR

